

03-FRS-Functional-Requirements-Specification

FRS — Functional Requirements Specification

Leish! v2 — Beauty Booking Marketplace

Table of Contents

1. Conventions
2. Actor Definitions
3. Functional Decomposition Map
4. FR-01 Authentication & Session
5. FR-02 Catalog & Discovery
6. FR-03 Booking Lifecycle
7. FR-04 Quotations
8. FR-05 Payments (Hybrid Deposit/Balance)
9. FR-06 Payouts & Commission
10. FR-07 Reviews & Ratings
11. FR-08 Messaging & Chat
12. FR-09 Profile Claim & Self-Service
13. FR-10 Admin Panel & Audit
14. FR-11 Email & Notifications
15. FR-12 Dashboard & UX
16. FR-13 Invoicing & Documents
17. FR-14 Compliance & Data Rights
18. FR-15 Platform Settings & Ops
19. Summary Table & MoSCoW
20. Appendix — State Machines & Flows

FRS — Functional Requirements Specification

Leish! v2 — Beauty Booking Marketplace

FIELD	VALUE
Document ID	LEISH-FRS-v2.0
Version	2.0.0
Date	2026-08-29
Status	Baseline
Predecessor	SRS (02), BRS (01)
Method	Use-case driven, MoSCoW prioritized
Notation	FR-XXX shall; pre/post conditions; primary/alternate flows

Table of Contents

1. [Conventions](#)
2. [Actor Definitions](#)
3. [Functional Decomposition Map](#)
4. [FR-01 Authentication & Session](#)
5. [FR-02 Catalog & Discovery](#)
6. [FR-03 Booking Lifecycle](#)
7. [FR-04 Quotations](#)
8. [FR-05 Payments \(Hybrid Deposit/Balance\)](#)
9. [FR-06 Payouts & Commission](#)
10. [FR-07 Reviews & Ratings](#)
11. [FR-08 Messaging & Chat](#)
12. [FR-09 Profile Claim & Self-Service](#)
13. [FR-10 Admin Panel & Audit](#)
14. [FR-11 Email & Notifications](#)
15. [FR-12 Dashboard & UX](#)
16. [FR-13 Invoicing & Documents](#)

- 17. [FR-14 Compliance & Data Rights](#)
- 18. [FR-15 Platform Settings & Ops](#)
- 19. [Summary Table & MoSCoW](#)
- 20. [Appendix — State Machines & Flows](#)

1. Conventions

- **ID:** `FR-<module>-<seq>` e.g. `FR-AUTH-01`.
- **Priority:** **Must** / **Should** / **Could** / **Won't** (MoSCoW).
- **Shall** = verifiable requirement; **Preconditions** must hold before execution; **Postconditions** hold after.
- **Source file** cites the authoritative implementation; tests must exercise it.
- **Acceptance criteria** written as Given/When/Then or checklist.

2. Actor Definitions

ACTOR	AUTH	ROUTE EXAMPLE	DETAILS
Guest	None	<code>/artists</code> , <code>/studios</code>	Can browse, search, view detail. Redirected to <code>/login?redirect=/dashboard</code> on gated actions.
Customer	JWT <code>customer</code>	<code>POST /api/bookings</code>	Books, pays, chats, reviews.
Artist	JWT <code>artist</code>	<code>PATCH /api/bookings/[id]</code>	Claims <code>artists.id</code> , manages bookings for claimed profiles.
Studio	JWT <code>studio</code>	same	Studio variant; scopes by <code>studio_id</code> when claimed.
Admin	JWT <code>admin</code>	<code>/api/admin/*</code>	Full ops; all mutations audited.
System	<code>CRON_SECRET</code> / internal	<code>POST /api/payments/webhook</code>	Billplz, cron, Agnost.

Role source: `src/lib/types.ts:1` (`Role`), DB `users.role CHECK (customer|artist|studio|admin)` `src/server/db.ts:212`.

3. Functional Decomposition Map

```
Leish v2
├── FR-01 Auth & Session
│   ├── Registration (role, consent, scrypt+pepper)
│   ├── Login / Logout / Session (JWT+JTI)
│   ├── Email verification (single-use, banner)
│   └── Password reset (forgot/reset, no enumeration)
├── FR-02 Catalog
│   ├── List/filter artists & studios
│   ├── Detail (slug→id resolution)
│   └── Admin + self-service edits
├── FR-03 Booking
│   ├── Create (price server-derived, slot-lock)
│   ├── List (scoped: owner vs claimed-artist)
│   └── Transitions (accept/reject/complete/cancel)
├── FR-04 Quotation (line-items, 24h TTL, re-quote)
├── FR-05 Payments (deposit, balance, webhook, refund)
├── FR-06 Payouts & Commission (compute, create, settle/fail)
├── FR-07 Reviews (gated, once-per-booking, atomic blend)
├── FR-08 Messaging (persist + SSE stream)
├── FR-09 Profile Claim (artist/studio scoping)
├── FR-10 Admin & Audit
├── FR-11 Email (provider abstraction, preferences, retries)
├── FR-12 Dashboard
├── FR-13 Invoicing (HTML + PDF)
├── FR-14 Compliance (export/delete)
└── FR-15 Settings & Ops (platform_settings, health, agnost)
```

4. FR-01 Authentication & Session

FR-AUTH-01 — REGISTRATION

FIELD	DETAIL
Pre	Guest, no active session for same email; <code>SESSION_SECRET</code> set in prod; <code>PEPPER_SECRET</code> optional
Input	<code>registerSchema</code> <code>src/server/validation.ts:5</code> — <code>name</code> 2–80, <code>email</code> lowercased RFC, <code>password</code> 8–128, <code>role</code> ∈ {customer,artist,studio}, <code>consent</code> bool, <code>consentTimestamp</code> ISO
Processing	Normalize email lower/trim; check <code>users.email</code> UNIQUE; hash with <code>scrypt</code> (salt 32B, keyLen 64, N=16384, r=8, p=1) + optional <code>PEPPER_SECRET</code> HMAC before hashing; <code>INSERT users (id, email, name, role, password, consent, consent_timestamp, created_at);</code> create <code>email_verifications</code> token (random, sha256 hash at rest); set <code>leish_session</code> cookie (see FR-AUTH-03); emit <code>agnost.begin(agentName="register")</code>
Post	User row created; <code>201</code> with <code>{user: toPublicUser, devVerifyUrl}</code> (dev); session cookie set; verification email queued (or outbox in dev)
Priority	Must
Source	<code>src/app/api/auth/register/route.ts</code> , <code>src/server/validation.ts</code> , <code>src/server/session.ts</code>
Acceptance	✓ Zod rejects weak/short name, invalid email, short password, invalid role. ✓ Duplicate email returns 409 without leaking enumeration (generic message). ✓ Password stored is not plaintext; <code>timingSafeEqual</code> used on verify. ✓ Response includes <code>devVerifyUrl</code> when <code>EMAIL_PROVIDER=dev</code>. ✓ Agnost <code>register</code> event emitted when <code>AGNOST_ORG_ID</code> set.

FR-AUTH-02 — LOGIN

FIELD	DETAIL
Input	<code>loginSchema</code> <code>src/server/validation.ts:14</code> — email + password (1–128)
Processing	Lookup <code>users</code> by lowercased email; script verify with pepper if set; on success rotate JTI: <code>INSERT sessions(jti,user_id,expires_at,created_at)</code> ; sign HS256 JWT <code>{sub,role,emailVerified, jti, iat, exp=iat+7d}</code> via <code>jose</code> ; set cookie; <code>agnost.begin(agentName="login")</code> end(true/false)
Post	<code>200 {user,}</code> + cookie; failed → <code>401</code> generic "Invalid credentials" (no enumeration)
Priority	Must
Source	<code>src/app/api/auth/login/route.ts</code>
Acceptance	✓ Correct creds → cookie present with <code>httpOnly; Path=/; SameSite=lax; Secure@prod</code> . ✓ Wrong password → 401 with no hint which field failed. ✓ Rate-limited after N attempts → 429 + <code>Retry-After</code> .

FR-AUTH-03 — SESSION COOKIE & VERIFICATION

FIELD	DETAIL
Shall	The system shall issue and verify the <code>leish_session</code> cookie as HS256 JWT per <code>src/server/session.ts</code> .
Processing	<code>createSessionToken({sub,role,...})</code> signs with <code>SESSION_SECRET</code> (must be ≥32 bytes in prod or throw). <code>verifySessionToken(token)</code> verifies signature via <code>jose.jwtVerify</code> , checks <code>exp</code> , then looks up <code>sessions</code> <code>WHERE jti=?</code> : if <code>revoked=1</code> → reject; if row missing → accept if sig+expiry pass (fail-open with warning); if <code>expires_at < now</code> → reject.
Priority	Must
Acceptance	✓ Token valid for 7 days (<code>SESSION_TTL_SECONDS = 604800</code>). ✓ Tampered payload → verification fails. ✓ Revoked JTI → rejected even if not expired. ✓ Missing JTI row → accepted on sig+expiry (infra degraded case).

FR-AUTH-04 — LOGOUT / REVOCATION

FIELD	DETAIL
Endpoint	POST /api/auth/logout
Processing	Extract <code>leish_session</code> from <code>Cookie</code> ; decode JTI without failing closed; <code>revokeSession(jti)</code> → <code>UPDATE sessions SET revoked=1 WHERE jti=?</code> ; clear cookie (<code>Max-Age=0</code>).
Priority	Must
Acceptance	✓ After logout, same token is rejected (revoked wins). ✓ Logout without session still clears cookie and returns 200.

FR-AUTH-05 — EMAIL VERIFICATION

FIELD	DETAIL
Endpoints	GET /api/auth/verify-email?token=... (redirect flow), POST /api/auth/resent-verification (authed, rate-limited)
Processing	On register: <pre>email_verifications(id,user_id,token_hash=sha256(token),expires_at~24h,used_at null) ; link {NEXT_PUBLIC_SITE_URL}/verify-email?token=raw</pre> On verify: hash lookup, check <code>used_at IS NULL && expires_at>now</code> , then <code>UPDATE users SET email_verified=1</code> + <code>UPDATE email_verifications SET used_at=now</code> . Unverified banner shown in <code>/dashboard</code> until verified with resend button (states idle/sending/sent).
Priority	Must
Acceptance	✓ Verification link single-use (second use fails). ✓ Expired token returns friendly error (not stack). ✓ Resend is rate-limited per user. ✓ Banner disappears immediately after verification. ✓ Payment routes can enforce <code>email_verified</code> when strict mode enabled (see FR-PAY-02).

FR-AUTH-06 — FORGOT / RESET PASSWORD

FIELD	DETAIL
Endpoints	POST /api/auth/forgot-password {email, turnstileToken?} , POST /api/auth/reset-password {token,newPassword}
Processing	password_resets(id,user_id,token_hash,expires_at~1h,used_at,created_at) . On forgot: rate-limit by IP; lookup user (always return 200-ish to avoid enumeration); if exists generate token, hash+store, send email (outbox in dev). On reset: verify hash, single-use, expiry, then script new password and mark used. PEPPER_SECRET applied if set.
Priority	Must
Acceptance	✓ Non-existent email returns success-shaped envelope (no enumeration). ✓ Token is single-use and 1h expiry. ✓ New password must meet 8–128 length. ✓ After reset, old password rejected. ✓ Dev response includes devResetUrl when provider is dev.

FR-AUTH-07 — ROLE HANDLING & GUARDS

FIELD	DETAIL
Shall	Every authed route shall resolve user from verified JWT + DB row; admin routes additionally call requireAdmin(request) (401 if unauthenticated, 403 if role $\neq$ admin) src/server/admin-auth.ts . Client AuthProvider (src/lib/auth.tsx) mirrors ROLE_LABELS for UX.
Acceptance	✓ /dashboard and /onboarding middleware redirect unauthenticated to /login . ✓ artist cannot hit /api/admin/* (403). ✓ requireAdmin is called first in every admin handler before any DB read.

5. FR-02 Catalog & Discovery

FR-CAT-01 — PUBLIC ARTIST LISTING

FIELD	DETAIL
Endpoint	<code>GET /api/artists?</code> <code>query=&state=&area=&bridal=&nonBridal=&budget=</code> (plus <code>/api/catalog/artists</code> wrapper for client components)
Processing	<code>artistsQuerySchema</code> <code>src/server/validation.ts:52</code> validates; <code>listArtists(filters)</code> <code>src/server/catalog.ts:198</code> does SQL <code>WHERE state=? AND area=? AND price_from ≤ ?</code> (indexed <code>idx_artists_state_area</code>), then <code>filterArtists()</code> for <code>query</code> (name/tagline/bio/specialties), <code>bridal</code> / <code>nonBridal</code> <code>≠ "any"</code> , and <code>date</code> available. <code>listAllArtists({limit,offset})</code> caps at 500.
Priority	Must
Acceptance	✓ Filter by Selangor returns only Selangor artists. ✓ Budget filters <code>price_from ≤ budget</code> when <code>budget > 0</code>. ✓ Empty result set returns <code>[]</code> not error. ✓ Pagination <code>{limit,offset}</code> respected and clamped (1-500).

FR-CAT-02 — ARTIST/STUDIO DETAIL RESOLUTION

FIELD	DETAIL
Shall	The system shall resolve an artist by slug first then by id: <code>resolveArtist(idOrSlug) = getArtistBySlug getArtistById</code> (<code>src/server/catalog.ts:256</code>). Same for studios (<code>resolveStudio</code>). <code>slug</code> equals legacy id for seeded rows (e.g. <code>aisha-azman</code>) so old links keep working.
Routing	<code>src/app/artists/[slug]/page.tsx</code> and <code>src/app/studios/[slug]/page.tsx</code> are <code>force-dynamic</code> (no stale static).
Acceptance	✓ <code>/artists/aisha-azman</code> resolves seeded artist. ✓ Unknown slug returns 404 via <code>notFound()</code> (not 200 shell — no root loading boundary). ✓ Admin-created artists with UUID <code>id</code> resolve by <code>slug</code> and <code>id</code>.

FR-CAT-03 — STATE→AREA CASCADING & VALIDATION

FIELD	DETAIL
Constants	<code>MALAYSIA_STATES</code> (16 entries), <code>AREAS_BY_STATE</code> mapping <code>src/lib/data.ts:3</code> , <code>BRIDAL_EVENTS</code> (5), <code>NON_BRIDAL_EVENTS</code> (6).
Processing	Client (<code>ArtistsBrowser</code>) cascades area list when state selected; server validates <code>state ∈ MALAYSIA_STATES</code> and <code>area string ≤80</code> ; <code>bridal ∈ BRIDAL_EVENTS.id</code> , <code>nonBridal ∈ NON_BRIDAL_EVENTS.id</code> .
Priority	Must
Acceptance	✓ Invalid <code>state</code> returns 400 zod error; area not required when state absent.

FR-CAT-04 — CATALOG MUTATIONS (ADMIN & SELF-SERVICE)

FIELD	DETAIL
Admin	<code>PATCH/POST /api/admin/artists</code> and <code>/api/admin/studios</code> via <code>updateArtist()/updateStudio()</code> with whitelists <code>ARTIST_UPDATE_FIELDS</code> / <code>STUDIO_UPDATE_FIELDS</code> <code>src/server/catalog.ts:305</code> ; JSON fields stringified; <code>updated_at=now</code> . <code>createArtist()/createStudio()</code> derive de-duplicated <code>slug</code> via <code>slugifyName()</code> , assign referral code, optionally link <code>referralCode</code> .
Self-service	<code>PATCH /api/artist-profiles</code> for claimed artists — same field whitelist minus <code>verified</code> / <code>referralEarnings</code> (audit).
Acceptance	✓ Unauthorized field names are ignored (not error, not written). ✓ Empty src uses <code>/images/hero.jpg</code> placeholder (prevents next/image crash). ✓ Mutations appear immediately due to <code>force-dynamic</code> (no CDN stale).

6. FR-03 Booking Lifecycle

FR-BOOK-01 — CREATE BOOKING

FIELD	DETAIL
Endpoint	POST /api/bookings — auth required
Input	bookingSchema src/server/validation.ts:19 — artistId (min1), service (min1), date YYYY-MM-DD regex, date ≥ today, time min1, notes ≤2000, eventType 1-80, venue ≤200, guestCount 0-1000
Processing	Resolve artist via getArtistById(artistId) else 404; verify service ∈ artist.services[].name else 400; construct BookingRow with artist_name=artist.name, price=serviceDef.price (server-derived), status='requested'; pre-check SELECT id FROM bookings WHERE artist_id=? AND date=? AND time=? AND status IN (requested,accepted,confirmed) → 409 if exists; then INSERT with race guard catching UNIQUE / duplicate key → 409; notifyBookingCreated() emails; Agnost booking- create begin/end; return serializeBooking()
Pre	User authed; if !email_verified, log info but allow (gate moved to payment); artistId exists; slot free
Post	Row bookings.id=randomUUID() created; response 201 {booking,user}
Priority	Must
Acceptance	✓ Client-sent price/duration ignored; server resolves from catalog. ✓ Past date rejected (zod refine). ✓ Concurrent double-book race returns 409 (never 500) — unique partial index is hard guarantee. ✓ Cancelled/completed slots don't block (WHERE status IN (...)).

FR-BOOK-02 — LIST BOOKINGS (SCOPED)

FIELD	DETAIL
Endpoint	GET /api/bookings?limit=&offset= — auth required
Processing	If <code>role ∈ (artist,studio)</code> : fetch <code>getClaimedArtistIds(user.id)</code> + <code>getClaimedStudioIds(user.id)</code> ; <code>claimed = studioIds.length>0 ? studioIds : artistIds</code> when <code>role=studio</code> ; <code>column = studioIds.length>0?"studio_id":"artist_id"</code> ; if <code>claimed.length==0</code> → <code>bookings=[]</code> ; else <code>SELECT * FROM bookings WHERE \${column} IN (?) ORDER BY date DESC,time DESC LIMIT ? OFFSET ?</code> plus count query. If customer: <code>WHERE user_id=?</code> . Serialize each via <code>serializeBooking()</code> (fetch deposit/balance payments, active quotation, balanceDueDate/balanceAmount). Pagination envelope <code>{bookings, pagination: {total,limit,offset,hasMore}}</code> . <code>limit</code> default 20 max 100.
Acceptance	✓ Artist sees only bookings for profiles they claimed (empty when none claimed). ✓ Customer sees only own <code>user_id</code> bookings. ✓ Studio with <code>studio_id</code> claim sees studio bookings; fallback to <code>artist_id</code> for legacy claims.

FR-BOOK-03 — BOOKING STATE TRANSITIONS

FIELD	DETAIL
Endpoint	PATCH /api/bookings/[id] {action: "accept" "reject" "complete" "cancel"} — auth required
State Machine	<pre>applyBookingTransition(current, action, {isOwner, role}) src/server/bookings.ts:40 :</pre> • <code>accept</code> : only artists, only from <code>requested</code> → <code>accepted</code> • <code>reject</code> : only artists, only from <code>requested</code> → <code>cancelled</code> • <code>complete</code> : only artists, only from <code>confirmed</code> → <code>completed</code> • <code>cancel</code> : from non-terminal (<code>!completed && !cancelled</code>), requires <code>isOwner artist</code> → <code>cancelled</code> <code>confirmOnFeePaid(current)</code> : only from <code>accepted</code> → <code>confirmed</code> (webhook only; not exposed via this PATCH).
Acceptance	✓ Artist <code>reject</code> on <code>accepted</code> returns error "Only requested..." ✓ Customer <code>accept</code> returns "Only artists..." ✓ Double <code>cancel</code> on <code>cancelled</code> returns "no longer be cancelled" ✓ Admin cannot bypass machine (admin uses separate <code>/api/admin/bookings</code> override).

FR-BOOK-04 — BOOKING SERIALIZATION & DERIVED FIELDS

FIELD	DETAIL
Shall	Every booking read shall be serialized via <code>serializeBooking(b: BookingRow)</code> <code>src/app/api/bookings/route.ts:32</code> : <code>getActiveQuotation(b.id)</code> , <code>getBookingFeeSen()</code> , <code>balanceDueDate = eventDate - 3d</code> (ISO), <code>totalPrice = quo</code> <code>max(0, total - fee)</code> . Return <code>{id, artistId, studioId, artistName, service, price, date, time, notes, status, eventType, venue, guestCount, quotat</code>
Acceptance	✓ <code>balanceAmount</code> never negative. ✓ Expired quotation yields <code>totalPrice=null</code> (not stale total). ✓ When no quotation, <code>totalPrice=null</code>.

7. FR-04 Quotations

FR-QUO-01 — CREATE / RE-QUOTE

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/quotation</code> — claimed artist only; booking must be <code>accepted</code>
Input	<code>quotationSchema</code> <code>src/server/validation.ts:34</code> — <code>baseFee</code> 0–10M sen int, <code>travelFee/earlyCallFee/accommodationFee</code> optional 0–10M, <code>extras</code> ≤10 items <code>{label 1-80, amount 0-10M}</code> , <code>artistNote</code> ≤1000
Processing	Validate ownership via claimed ids; check booking <code>status==='accepted'</code> ; <code>quotationTotal(input)</code> sums all line items; supersede: <code>UPDATE quotations SET status='superseded' WHERE booking_id=? AND status='pending'</code> ; create row <code>id=randomUUID()</code> , <code>status='pending'</code> , <code>created_at=now</code> , <code>expires_at=now+24h</code> ; <code>bind(row)</code> insert; notify <code>notifyQuotationSent</code> email; Agnost <code>quotation</code> event.
Post	<code>201 {quotation: serializeQuotation(row)}</code>
Acceptance	✓ Amounts outside 0–10M rejected. ✓ 11th extra rejected (max 10). ✓ Previous <code>pending</code> becomes <code>superseded</code> atomically before insert. ✓ Unknown booking 404; not-claimed 403; wrong status 400.

FR-QUO-02 — EXPIRY SEMANTICS

FIELD	DETAIL
Shall	Per <code>src/server/quotations.ts:13</code> , <code>QUOTATION_TTL_MS=24h</code> . <code>isQuotationExpired(row)</code> checks <code>status==='pending' && expires_at < now</code> . <code>getActiveQuotation(bookingId)</code> lazily marks expired to <code>'expired'</code> on read; <code>findExpiredQuotations()</code> + <code>markQuotationExpired()</code> support cron sweep. Client shows countdown from <code>createdAt/expiresAt</code> ; <code>totalPrice</code> nulled when expired; artist sees "Send new quotation" when <code>accepted && expired</code> .
Acceptance	✓ Quotation older than 24h reads as <code>expired</code> and stays <code>expired</code>. ✓ Re-quote after expiry succeeds (supersede not needed when previous is <code>expired</code>).

8. FR-05 Payments (Hybrid Deposit/Balance)

FR-PAY-01 — PAYMENT TYPES & SCHEMA

FIELD	DETAIL
Schema	<code>payments(id TEXT PK, booking_id FK ON DELETE CASCADE, type TEXT CHECK(deposit,balance) DEFAULT deposit, amount INTEGER, currency MYR, provider dev billplz, status required paid failed refunded, provider_ref, provider_url, created_at, updated_at)</code> with <code>UNIQUE(booking_id,type)</code> <code>uq_payments_booking_type</code> and <code>CASCADE</code> semantics <code>src/server/db.ts:318</code> . <code>PaymentRecord</code> type <code>src/server/payments.ts:32</code> .
Invariants	One deposit and at most one balance per booking, never two of same type (DB unique). <code>amount</code> stored in sen. <code>provider_url</code> is Billplz hosted page URL (nullable).
Acceptance	✓ Duplicate <code>INSERT (booking_id,type)</code> violates unique index. ✓ Deleting booking cascades delete payments.

FR-PAY-02 — CREATE DEPOSIT PAYMENT (PAY-FEE)

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/pay-fee {turnstileToken?}</code> — customer who owns booking; booking <code>status==='accepted'</code> ; quotation <code>pending</code> or already expired? (re-quote path).
Processing	Verify ownership + <code>email_verified</code> (gate at payment, see <code>src/app/api/bookings/route.ts:161</code> comment — unverified allowed at request but blocked or warned at fee); <code>getBookingFeeSen()</code> default 5000 sen; check no existing <code>payments(type=deposit,status IN (required,paid))</code> already; call <code>createBookingPayment(bookingId,'deposit',amountSen)</code> which branches on <code>activePaymentProvider()</code> (<code>BILLPLZ_API_KEY && COLLECTION_ID ? billplz:dev</code>). <code>billplzPayment</code> : fetch owner name/email, POST <code>{BILLPLZ_API}/bills</code> with Basic auth, fields <code>collection_id,name,email,mobile,amount,description,callback_url,redirect_url,reference_1=bookingId</code>; parse <code>{id,url}</code> ; <code>insertPayment({... status='required',provider='billplz',provider_ref=id,provider_url=url})</code> ; return <code>{id,amount,provider,reference,url}</code> . <code>devPayment</code> : synthesize <code>provider_ref=dev_...</code> , insert <code>status='required'</code> . For dev/e2e, immediately <code>markBillPaid(billId)</code> + <code>handlePaymentPaid</code> to auto-confirm. Rate-limit + optional Turnstile verify.
Responses	<code>200 {payment:{amount,type,status,provider,reference,url}}</code> plus <code>booking:{status}</code> after dev auto-settlement (<code>confirmed</code>).
Acceptance	✓ Amount equals <code>getBookingFeeSen()</code> (not client-provided). ✓ Billplz failure (<code>!res.ok !body.id</code>) throws “Failed to create payment” and logs (<code>pino.error</code>). ✓ Dev path is synchronous await (row durable before webhook reads). ✓ Second <code>pay-fee</code> on same booking returns 409 (“deposit already required/paid”).

FR-PAY-03 — CREATE BALANCE PAYMENT (PAY-BALANCE)

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/pay-balance</code> — owner, booking <code>status==='confirmed'</code> , active quotation <code>total</code> exists and not <code>expired</code> .
Processing	Mirror <code>pay-fee</code> but <code>type='balance'</code> and <code>amount = max(0, quotation.total - bookingFeeSen)</code> . If <code>balanceAmount===0</code> → return informative error or treat as no-op paid. Create <code>payments(type=balance)</code> via <code>createBookingPayment</code> . Dev auto-settles similarly and triggers <code>handlePaymentPaid</code> → <code>createPayoutForBooking</code> .
Acceptance	✓ Amount derived (<code>quotation.total - deposit</code>) never client-supplied. ✓ No active quotation or expired → 400. ✓ 0 balance edge case handled (commission + fee covers total).

FR-PAY-04 — BILLPLZ WEBHOOK

FIELD	DETAIL
Endpoint	<code>POST /api/payments/webhook</code> — public, but HMAC-protected
Processing	Read raw body string (NOT parsed JSON) for <code>verifyBillplzSignature(rawBody, X-Billplz-Signature, API_KEY)</code> <code>src/server/payments.ts:381</code>: check header present, hex64 regex, <code>HMAC-SHA256(apiKey, rawBody).hex</code>, <code>timingSafeEqual</code> buffers; if false → <code>401</code> and no state change. On pass: parse JSON fields, <code>billId = body.id</code>, <code>getPaymentForBill(billId)</code> else <code>404</code>; <code>markBillPaid(billId)</code> (<code>UPDATE payments SET status='paid' where provider_ref=billId</code>); <code>handlePaymentPaid(payment)</code>: if <code>deposit</code> → <code>confirmOnFeePaid(booking.status)</code> → <code>UPDATE bookings SET status='confirmed'</code> if ok; if <code>balance</code> → <code>UPDATE quotations SET status='paid'</code> + <code>createPayoutForBooking(...)</code>. Idempotent (re-delivery no-op). Log via <code>pino</code>.
Acceptance	✓ Missing/bad signature → <code>401</code>; no DB write. ✓ Forged <code>paid:true</code> without valid HMAC is ignored. ✓ Replay of same bill id is idempotent (second <code>markBillPaid</code> still true but status stays paid). ✓ Signature over raw body (not pretty-printed JSON) — test with exact byte string.

FR-PAY-05 — REFUND (BALANCE ONLY)

FIELD	DETAIL
Endpoint	<code>POST /api/bookings/[id]/refund</code> — customer who owns booking, or admin; booking <code>status==='cancelled'</code>
Processing	<code>getPaymentForBooking(bookingId, 'balance')</code>; guard <code>type==='balance'</code> else "Only balance..."; guard <code>status==='paid'</code> else "Only paid balances..."; <code>amountSen=payment.amount</code>; if <code>provider==='billplz' && BILLPLZ_API_KEY && provider_ref</code> → <code>POST \$ {BILLPLZ_API}/bills/{provider_ref}/refund {amount, reason:"Booking cancelled"}</code> — on failure throw; then <code>UPDATE payments SET status='refunded'</code>. <code>provider='dev'</code> just marks refunded. Return updated payment.
Acceptance	✓ Deposit row never refunded (guard). ✓ Unpaid balance → error. ✓ Billplz refund failure rolls back (not marked refunded).

9. FR-06 Payouts & Commission

FR-PAYO-01 — COMMISSION COMPUTATION

FIELD	DETAIL
Pure Function	<pre>computeCommission(totalSen, rateBps, waiverSen) → {totalSen, waived, commissionSen, artistNetSen} src/ server/settings.ts:100 — total=max(0,round(totalSen)), waived=total<waiver, commission=waived?0:round(total*rate/10000), artistNetSen=total-commission.</pre>
Settings	<pre>getCommissionRateBps() clamped 0–5000; getCommissionWaiverSen() ≥0; DEFAULT_COMMISSION_RATE_BPS=1000, DEFAULT_COMMISSION_WAIVER_SEN=10000 src/server/ settings.ts:21 ; 30s cache src/server/settings.ts:25.</pre>
Acceptance	✓ <code>computeCommission(50_00,1000,10000)</code> → waived true, commission 0. ✓ <code>computeCommission(100000,1000,10000)</code> → 10000 commission. ✓ Negative inputs clamped to 0. ✓ Rate 6000 clamped to 5000.

FR-PAYO-02 — PAYOUT CREATION

FIELD	DETAIL
Function	<pre>createPayoutForBooking(bookingId, {artistId,eventDate,quoteTotalSen}) src/server/payouts.ts:56 — idempotent: if SELECT * FROM payouts WHERE booking_id=? exists return it; else fetch rateBps,waiverSen,depositSen in parallel; breakdown=computeCommission(...); artistReceives=max(0, breakdown.artistNetSen - depositSen); settleableAt = eventDate? eventDateT00:00+24h ISO : null; artistUserId = SELECT user_id FROM artist_profiles WHERE artist_id=? else null; INSERT payouts(id,artist_user_id,booking_id,gross,commission,net,status=pending,settleable_at,settled_at null, notes=waived?"Commission waived":null, created_at); pino.info.</pre>
Trigger	Called only from <code>handlePaymentPaid</code> when <code>type==='balance'</code> is paid (never from deposit).
Acceptance	✓ Second call with same bookingId returns existing row (no duplicate). ✓ <code>net = gross - commission - deposit</code> (e.g. 100k-10k-5k=85k). ✓ Unclaimed artist → <code>artist_user_id=null</code> still creates payout. ✓ <code>settleable_at</code> exactly 24h after event date midnight UTC.

FR-PAYO-03 — PAYOUT ADMINISTRATION

FIELD	DETAIL
Endpoints	<code>GET /api/admin/payouts?status=</code> (optional filter), <code>PATCH /api/admin/payouts {payoutId, status:'settled' 'failed', notes?}</code> — admin only
Processing	<code>listPayouts(status?)</code> joins <code>payouts</code> JOIN <code>bookings</code> for display; <code>updatePayoutStatus(id, status, notes)</code> does <code>UPDATE payouts SET status=?, settled_at=(settled? now:null), notes=COALESCE(?,notes) WHERE id=?</code> ; returns row or 404. Caller (<code>/api/admin/payouts</code> route) calls <code>logAdminAction</code> after.
Acceptance	✓ Non-admin → 403. ✓ Missing payout → 404. ✓ Settled row gets <code>settled_at</code> ISO; <code>failed</code> gets <code>settled_at null</code>.

10. FR-07 Reviews & Ratings

FR-REV-01 — ADD REVIEW

FIELD	DETAIL
Endpoint	<code>POST /api/artists/[slug]/reviews {rating 1-5, text, event?}</code> — auth required? (customer)
Types	<code>Review</code> <code>src/lib/types.ts:15</code> <code>{id,author,rating,date,text,event}</code> ; <code>NewReviewInput</code> <code>src/server/catalog.ts:580</code> adds <code>entityType,entityId,bookingId,userId</code>
Processing	<code>addEntityReview(input)</code> <code>src/server/catalog.ts:633</code> : if <code>bookingId</code> provided, guard <code>SELECT id FROM reviews WHERE booking_id=?</code> → throw <code>ALREADY_REVIEWED</code> if exists; <code>INSERT reviews(id,entity_type,entity_id,booking_id,user_id,author_name,rating,event,text,created_at)</code> with <code>rating=round(rating)</code> ; then <code>blendAggregate(entityType,entityId,rating)</code> ; fetch back and map via <code>rowToPublicReview</code> .
Acceptance	✓ Duplicate <code>bookingId</code> review → error <code>ALREADY_REVIEWED</code> (409). ✓ Rating rounded to int server-side. ✓ Review appears via <code>listEntityReviews(entityType,entityId)</code> ordered <code>created_at DESC</code>.

FR-REV-02 — GATE REVIEW ELIGIBILITY

FIELD	DETAIL
Function	<pre>findReviewableBooking(userId, entityType, entityId)</pre> <code>src/server/catalog.ts:678</code>: currently only <code>entityType==='artist'</code> supported (studio returns null). Query: <pre>SELECT b.id FROM bookings b LEFT JOIN reviews r ON r.booking_id=b.id WHERE b.user_id=? AND b.artist_id=? AND b.status='completed' AND r.id IS NULL ORDER BY b.date DESC LIMIT 1.</pre>
Acceptance	✓ Completed booking without review → row returned. ✓ Pending/accepted/confirmed booking → null. ✓ Already-reviewed booking → null (JOIN finds r.id). ✓ Studio entity → null (explicit).

FR-REV-03 — ATOMIC RATING BLEND

FIELD	DETAIL
Function	<pre>blendAggregate(entityType, entityId, rating)</pre> <code>src/server/catalog.ts:615</code>: <code>UPDATE artists studios SET review_count=review_count+1, rating=ROUND(((rating*review_count)+?)/(review_count+1),2), updated_at=now WHERE id=?</code> — single statement.
Acceptance	✓ Concurrent reviews don't lose updates (no read-modify-write). ✓ Rating rounded 2dp via <code>ROUND(...,2)</code> (both PG + SQLite). ✓ New <code>review_count</code> is old+1, new rating correctly weighted.

11. FR-08 Messaging & Chat

FR-MSG-01 — PERSIST MESSAGES

FIELD	DETAIL
Table	<code>messages(id TEXT PK, booking_id FK CASCADE, sender_id FK CASCADE, body TEXT, created_at TEXT)</code>
Endpoints	<code>GET /api/bookings/[id]/messages</code> (participants only) and <code>POST /api/bookings/[id]/messages {body}</code>
Processing	Authz: <code>user_id == booking.user_id</code> OR <code>artist_id ∈ claimedArtists/user</code> ; if not participant → 403. POST sanitizes <code>body</code> trim, non-empty, ≤2000?; <code>INSERT messages(randomUUID, booking_id, sender_id=user.id, body, created_at=now)</code> ; publish to <code>chat-bus</code> for SSE. GET returns ordered <code>created_at ASC</code> .
Acceptance	✓ Third user who is neither owner nor claimed artist is blocked (403). ✓ Empty body rejected 400. ✓ Message appears immediately on GET.

FR-MSG-02 — LIVE STREAMING (SSE)

FIELD	DETAIL
Endpoint	<code>GET /api/bookings/[id]/messages/stream</code> — <code>text/event-stream</code>
Processing	Authz same as FR-MSG-01; handler replays history <code>(SELECT * FROM messages WHERE booking_id=? ORDER BY created_at ASC)</code> as <code>data: JSON{ id, senderName, body, createdAt }</code> events, then subscribes to <code>src/server/chat-bus.ts</code> in-memory pub/sub for new messages on that booking channel; keepalive every 30s.
Acceptance	✓ New browser joining stream receives history then live frames. ✓ Disconnect + reconnect replays full history (no lost messages). ✓ Connection is authenticated (cookie same as HTTP routes).

12. FR-09 Profile Claim & Self-Service

FR-CLAIM-01 — CLAIM A CATALOG PROFILE

FIELD	DETAIL
Endpoint	<code>POST /api/artist-profiles {artistId}</code> — role must be <code>artist studio</code> ; one claim per <code>user_id</code> .
Table	<code>artist_profiles(user_id PK FK users, artist_id UNIQUE, claimed_at)</code> + parallel <code>studio_profiles</code>
Processing	Check <code>role ∈ (artist,studio)</code> else 403; validate <code>artistId</code> exists in <code>artists</code> (or studios for studio role); check not already claimed globally (<code>uq_artist_profiles_artist_id</code>); then <code>INSERT</code> and return <code>profile</code> . Emits claim event for admin.
Acceptance	✓ Customer role cannot claim (403). ✓ Already claimed <code>artist_id</code> returns 409. ✓ Second claim by same <code>user_id</code> violates PK → 409 (one profile per account).

FR-CLAIM-02 — FETCH CLAIMED PROFILE

FIELD	DETAIL
Endpoint	<code>GET /api/artist-profiles</code> — authed
Processing	<code>SELECT artist_id FROM artist_profiles WHERE user_id=?</code> (and studio equivalent); if studio role prefer <code>studio_profiles</code> fallback to <code>artist_profiles</code> (shared table history). Return <code>profile:null</code> when none claimed; else <code>{artistId, artistName}</code> .
Acceptance	✓ Customer with no claim → <code>{profile:null}</code> (not 404). ✓ Claimed artist sees own name.

FR-CLAIM-03 — SELF-SERVICE PROFILE EDITS

FIELD	DETAIL
Endpoint	<code>PATCH /api/artist-profiles {updates}</code> — claimed artist only
Processing	Load claimed <code>artistId</code> for caller; apply via <code>updateArtist(artistId, updates)</code> with whitelisted <code>ARTIST_UPDATE_FIELDS</code> subset for self-service (exclude <code>verified</code> / <code>referralEarnings</code>). Validate fields server-side (e.g. <code>priceFrom</code> 0–10M). Update <code>updated_at</code> .
Acceptance	✓ Unclaimed artist → 403. ✓ Field <code>verified</code> silently ignored (cannot self-verify). ✓ Admin edits via <code>/api/admin/artists</code> can set all fields including <code>verified</code> .

13. FR-10 Admin Panel & Audit

FR-ADM-01 — GUARD

FIELD	DETAIL
Server guard	<code>requireAdmin(request)</code> <code>src/server/admin-auth.ts</code> — extracts <code>leish_session</code> , <code>verifySessionToken</code> , loads <code>users</code> row, checks <code>role==='admin'</code> ; returns <code>401</code> <code>jsonError("Not authenticated")</code> or <code>403</code> <code>jsonError("Forbidden")</code> .
Layout guard	<code>src/app/admin/layout.tsx</code> does same server-side before render and redirects non-admins; <code>AdminShell</code> provides sidebar + drawer.
Seed	<code>scripts/seed-admin.ts</code> idempotent: <code>ADMIN_EMAIL/ADMIN_PASSWORD</code> creates user with <code>role='admin'</code> or upgrades existing <code>users.role</code> to admin.
Acceptance	✓ Non-admin direct API call blocked (not just UI hidden). ✓ Audit of failed admin attempt not required but recommendation logged.

FR-ADM-02 — ADMIN CRUD (OVERVIEW)

RESOURCE	ENDPOINTS (UNDER <code>SRC/APP/API/ADMIN/*</code>)	ALLOWED OPERATIONS	AUDIT
Dashboard	<code>GET /api/admin</code>	Read aggregates: users by role, bookings by status, payments total/paid/required/revenue, <code>artistProfiles</code> count, recentBookings (10), recentAudit (10)	No
Users	<code>GET/POST /api/admin/users</code> , <code>PATCH/DELETE /api/admin/users/[id]</code>	Full CRUD (never expose password hash)	Yes — every mutation
Artists	<code>GET/POST /api/admin/artists</code> , <code>PATCH /api/admin/artists/[id]</code>	Create via <code>createArtist</code> , update via <code>updateArtist</code> (all whitelisted fields + <code>verified</code>)	Yes
Studios	Mirrors Artists	Same via <code>createStudio</code> / <code>updateStudio</code>	Yes
Bookings	<code>GET /api/admin/bookings</code> , <code>PATCH /api/admin/bookings/[id]</code>	Admin status override + notes	Yes
Payments	<code>GET /api/admin/payments</code>	List + filter by booking/user	No
Payouts	<code>GET/PATCH /api/admin/payouts</code>	See FR-PAYO-03	Yes
Quotations	<code>GET /api/admin/quotations</code>	List + expire sweep trigger	No
Messages	<code>GET /api/admin/messages</code>	Read all booking threads	No
Emails	<code>GET /api/admin/emails</code>	View <code>email_outbox</code> + retries	No
Audit	<code>GET /api/admin/audit</code>	List <code>admin_audit_log</code> ordered desc	No
Settings	<code>GET/PATCH /api/admin/settings</code>	<code>platform_settings</code> CRUD	Yes
Analytics	<code>GET /api/admin/analytics</code>	Agnost rollups if configured	No

Conventions — Admin pages (`src/app/admin/*`) are "use client" and fetch from `/api/admin/*` via `.then()` chains (never synchronous `setState` in effect body per `react-hooks/set-state-in-effect` rule).

FR-ADM-03 — AUDIT LOG

FIELD	DETAIL
Table	<code>admin_audit_log(id TEXT PK, admin_user_id FK users ON DELETE SET NULL, action TEXT, target_table TEXT, target_id TEXT, details TEXT JSON default '{}', created_at TEXT)</code> with indexes <code>idx_audit_log_admin</code> , <code>idx_audit_log_created</code> .
Helper	<code>logAdminAction({adminUserId, action, targetTable, targetId, details})</code> — called after every successful admin mutation; failure to log shall not roll back the mutation but shall emit <code>pino.error</code> .
FK Note	<code>admin_user_id</code> has real FK to <code>users(id)</code> — integration tests must seed a user row before writing audit entries; otherwise foreign key violation.
Acceptance	✓ Admin <code>PATCH /api/admin/artists/[id]</code> creates audit row with <code>action='update'</code> + <code>target_table='artists'</code>. ✓ Deleting the admin user sets <code>admin_user_id</code> to null (<code>SET NULL</code>) not cascade delete.

FR-ADM-04 — SETTINGS ADMINISTRATION

FIELD	DETAIL
Keys	<code>booking_fee_sen</code> , <code>commission_rate_bps</code> , <code>commission_waiver_sen</code> (+ extensible others like <code>platform_name</code> , <code>support_email</code> may be added).
Endpoint	<code>PATCH /api/admin/settings {key,value}</code> — validate <code>value</code> is numeric within allowed ranges; write <code>platform_settings(value, updated_by=adminUserId, updated_at=now)</code> with <code>INSERT ... ON CONFLICT(key) DO UPDATE</code> . Invalidate cache <code>clearSettingsCache()</code> effect via next read (30s TTL expiry).
Acceptance	✓ Invalid key/value returns 400 validation error. ✓ Changes take effect within 30s (cache TTL) without redeploy. ✓ Setting read failure always falls back to default (never breaks payments).

14. FR-11 Email & Notifications

FR-MAIL-01 — PROVIDER ABSTRACTION

FIELD	DETAIL
Module	<code>src/server/email.ts</code> <code>sendEmail({to, subject, text, html})</code> — selects <code>EMAIL_PROVIDER</code> env (default <code>dev</code>): <code>dev</code>: <code>INSERT email_outbox(id,to_email,subject,text,html,created_at)</code> and return dev link. <code>resend</code>: POST Resend API with <code>RESEND_API_KEY</code>, <code>EMAIL_FROM</code>. <code>postmark</code>: POST Postmark with <code>POSTMARK_SERVER_TOKEN</code>. <code>brevo</code>: POST Brevo with <code>BREVO_API_KEY</code>. Missing creds for selected provider → fallback to dev outbox + <code>pino.warn</code> (never silent).
Acceptance	✓ Unknown <code>EMAIL_PROVIDER</code> falls back to dev. ✓ <code>EMAIL_FROM</code> default is <code>Leish! <no-reply@leish.my></code>. ✓ Dev outbox viewable at <code>/dev/emails</code> (only non-production).

FR-MAIL-02 — BOOKING EMAILS

FIELD	DETAIL
Module	<code>src/server/booking-emails.ts</code> composes all transactional copy: <code>notifyBookingCreated</code>, <code>notifyQuotationSent</code>, <code>notifyInvoiceSent</code>, <code>notifyQuotationExpiry</code>, <code>notifyBalanceReminder</code>, <code>notifyStatusChanged</code>.
Acceptance	✓ New booking triggers <code>booking_created</code> email to relevant party. ✓ Quotation triggers <code>quotation_sent</code>. ✓ Balance reminder (<code>POST /api/bookings/[id]/remind</code> from confirmed bookings) respects <code>email_preferences.balance_reminder</code>.

FR-MAIL-03 — PREFERENCES & RETRIES

FIELD	DETAIL
Preferences	<code>email_preferences(user_id PK FK, booking_created, quotation_sent, invoice_sent, quotation_expiry, balance_reminder, status_changed INTEGER 0/1, updated_at)</code> default <code>1</code> for each; <code>PATCH /api/email/preferences</code> upserts.
Retries	<code>email_retries(id,to_email,subject,text,html,attempts,max_attempts=3,next_retry,last_error,created_at)</code> — on send failure, row inserted with <code>next_retry = now + backoff</code>; cron retries via <code>src/app/api/cron/*</code> guarded by <code>CRON_SECRET</code>.
Acceptance	✓ User can toggle each preference off and stop receiving that class. ✓ Retry attempts capped at 3, then dropped with logged error.

15. FR-12 Dashboard & UX

FR-DASH-01 — CLIENT DASHBOARD

FIELD	DETAIL
Route	<code>/dashboard</code> — client-only (<code>"use client"</code>), requires auth else CTA "Please sign in".
Data	Fetches <code>GET /api/bookings</code> + <code>GET /api/catalog/artists</code> + <code>GET /api/artist-profiles</code> in effects with <code>.then()</code> chains; shows banners (verify email), stats (customer stats hidden; MUA stats: requests count, quotations open, confirmed, earnings paid/pending).
Views	For each booking card: avatar, service · price · eventType, status badges (<code>requested</code> amber, <code>accepted</code> sky, <code>confirmed</code> emerald, <code>completed</code> stone, <code>cancelled</code> red), payment badge, fee <code>fee {status}</code> , quotation block (lines + totals + 24h expiry), chat thread, actions: cancel / pay-fee / pay-balance / refund / invoice / send-reminder (role-aware). Invoices link to HTML + PDF.
Acceptance	✓ Unverified customer sees amber banner with resend (idle/sending/sent). ✓ <code>GET /api/bookings</code> failure shows empty state with correct CTA per role. ✓ Stats compute correctly from <code>bookings + payments + quotations</code>.

FR-DASH-02 — ARTIST CLAIM UI

FIELD	DETAIL
Block	Visible when <code>role ∈ (artist,studio)</code> ; reads <code>GET /api/artist-profiles</code> ; if null shows <code><select></code> populated from <code>/api/catalog/artists</code> and <code>POST /api/artist-profiles</code> on submit; after claim shows "You are managing "...". + refresh.
Acceptance	✓ Claimed profile id shown correctly. ✓ Select disabled until catalog loaded. ✓ Claim error surfaced as <code>role="alert"</code>.

16. FR-13 Invoicing & Documents

FR-INV-01 — PRINTABLE INVOICE (HTML) & PDF

FIELD	DETAIL
Endpoints	<code>GET /api/bookings/[id]/invoice</code> (HTML) and <code>GET /api/bookings/[id]/invoice.pdf</code> (PDF via <code>pdf-lib</code>)
Authz	Owner or claimed artist/studio only; same guard as messaging.
Content	Header (Leish! branding), billing to client, artist + service + date/time + venue, quotation line items (base/travel/early-call/accommodation + extras), <code>booking_fee_sen</code> line, total, payment status per type, commission note when relevant. PDF rendered server-side (no browser print-sim needed).
Acceptance	✓ Unauthorized returns 403. ✓ Invoice reflects server-computed <code>quotation.total</code>, not cached client price. ✓ PDF downloads with correct <code>Content-Disposition</code>.

17. FR-14 Compliance & Data Rights

FR-COMP-01 — DATA EXPORT

FIELD	DETAIL
Endpoint	<code>GET /api/me/export</code> — authed
Processing	Aggregates <code>users</code> public fields, <code>bookings</code> , <code>messages</code> , <code>payments</code> , <code>reviews</code> , <code>artist_profiles</code> , <code>email_preferences</code> ; returns <code>application/json</code> blob with <code>Content-Disposition: attachment</code> named <code>leish-data-<id>.json</code> ; used by <code>/dashboard</code> Export button.
Acceptance	✓ Result is valid JSON containing all owned records. ✓ No other users' data leaked.

FR-COMP-02 — ACCOUNT DELETION

FIELD	DETAIL
Endpoint	<code>DELETE /api/me?confirm=1</code> — authed, <code>confirm</code> required to avoid accidental hit
Processing	Confirm <code>confirm=1</code> ; cascade delete via FK <code>ON DELETE CASCADE</code> for bookings/payments/quotations/messages/sessions; <code>ON DELETE SET NULL</code> for audit entries; remove user row; clear cookie; return 200. Dashboard confirm dialog requires explicit user confirmation.
Acceptance	✓ Without <code>confirm=1</code> returns 400. ✓ After delete, login with same email fails (user gone). ✓ Audit log retains entry with <code>admin_user_id=null</code> (not lost).

18. FR-15 Platform Settings & Ops

FR-OPS-01 — SETTINGS RUNTIME

FIELD	DETAIL
Table	<code>platform_settings(key TEXT PK, value TEXT, updated_by FK, updated_at TEXT)</code>
Accessors	<code>getSetting/getNumberSetting/getBookingFeeSen/getCommissionRateBps/getCommissionWaiverSen/clearSettingsCache</code> <code>src/server/settings.ts</code> . Cache 30s process-local.
Acceptance	✓ Cache clears after <code>clearSettingsCache()</code> (used in tests). ✓ Read failure never throws — falls back to default and warns.

FR-OPS-02 — HEALTH & CRON

FIELD	DETAIL
Health	<code>GET /api/health</code> → <code>{ok:true, db: "postgres" "sqlite", env: "production" ...}</code> ; Docker HEALTHCHECK <code>http://localhost:3000/api/health</code> .
Cron	<code>GET/POST /api/cron/*</code> (e.g. <code>expire-quotations</code> , <code>retry-emails</code>) guarded by <code>CRON_SECRET</code> bearer; Vercel Cron or external scheduler calls them.
Acceptance	✓ Health returns 200 even when <code>DATABASE_URL</code> unset (sqlite mode). ✓ Cron without valid secret returns 401.

FR-OPS-03 — ANALYTICS (AGNOST AI)

FIELD	DETAIL
Server	<code>src/instrumentation.ts</code> initializes <code>agnostai</code> SDK with <code>AGNOST_ORG_ID</code> ; every key route wraps with <code>agnost.begin({userId, agentName, input})</code> → <code>interaction.end(output, success)</code> : register, login, booking-create, booking-status update, quotation, payment webhook, profile edit.
Client	<code>src/lib/agnost-client.ts</code> <code>trackEvent</code> , <code>trackArtistView</code> , <code>trackSearch</code> , <code>trackBookingForm</code> called from <code>ArtistsBrowser</code> and booking/calendar components when <code>NEXT_PUBLIC_AGNOST_ORG_ID</code> set.
Dashboard	<code>https://app.agnost.ai/projects/3e27e121-654d-4746-ba55-7996f21bb351</code>
Acceptance	✓ When <code>AGNOST_ORG_ID</code> missing, instrumentation is no-op (no crash). ✓ Tracking never leaks PII beyond <code>userId/bookingId</code> (hashed where needed).

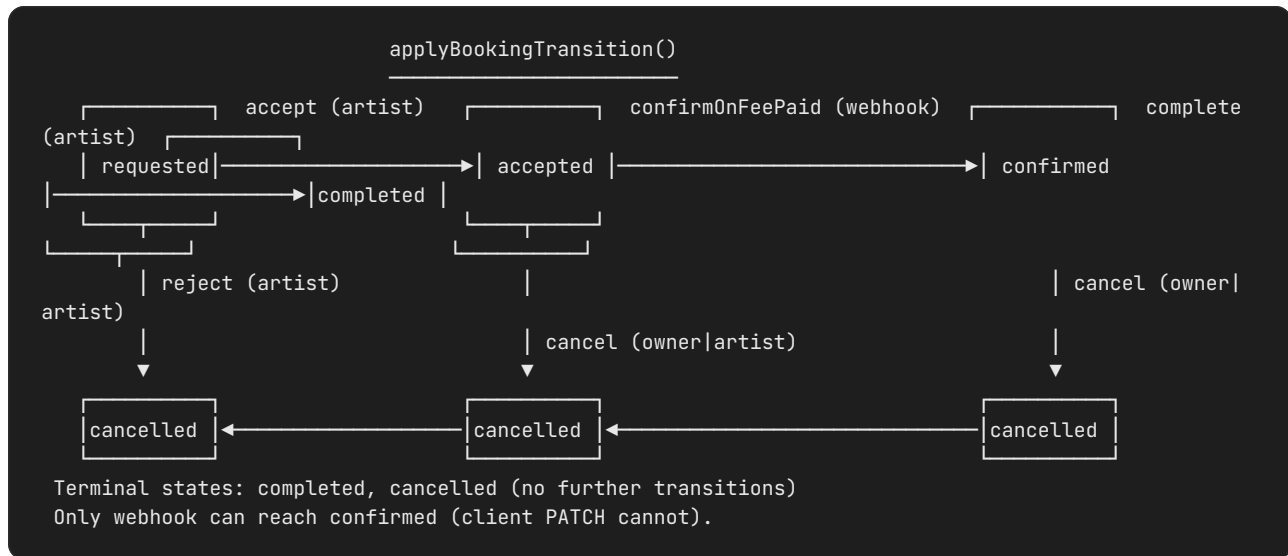
19. Summary Table & MoSCoW

MODULE	FR IDS	COUNT	MUST	SHOULD	COULD
Auth	FR-AUTH-01..07	7	7	—	—
Catalog	FR-CAT-01..04	4	4	—	—
Booking	FR-BOOK-01..04	4	4	—	—
Quotation	FR-QUO-01..02	2	2	—	—
Payments	FR-PAY-01..05	5	5	—	—
Payouts	FR-PAYO-01..03	3	3	—	—
Reviews	FR-REV-01..03	3	3	—	—
Messaging	FR-MSG-01..02	2	2	—	—
Profile Claim	FR-CLAIM-01..03	3	3	—	—
Admin	FR-ADM-01..04	4	4	—	—
Email	FR-MAIL-01..03	3	2	1	—
Dashboard	FR-DASH-01..02	2	1	1	—
Invoicing	FR-INV-01	1	—	1	—
Compliance	FR-COMP-01..02	2	2	—	—
Ops	FR-OPS-01..03	3	2	1	—
Total		48	42	4	—

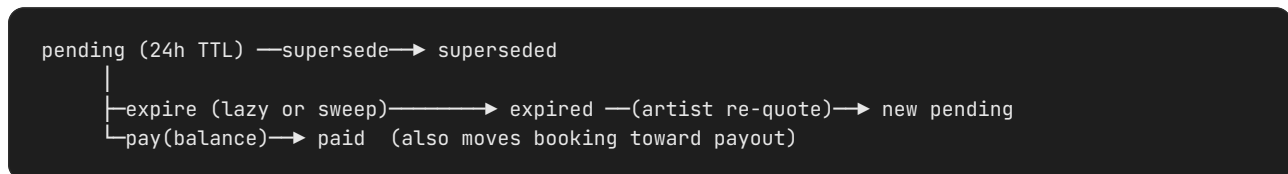
All **Must** items constitute the launch gate. **Should** items are highly desired for launch but may slip to v2.1 without blocking.

20. Appendix — State Machines & Flows

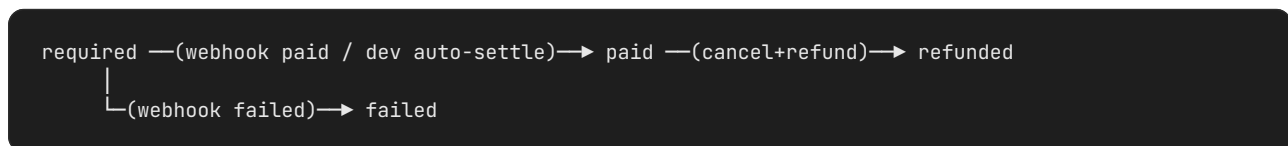
20.1 BOOKING STATUS STATE MACHINE (FROM `SRC/SERVER/BOOKINGS.TS:1`)



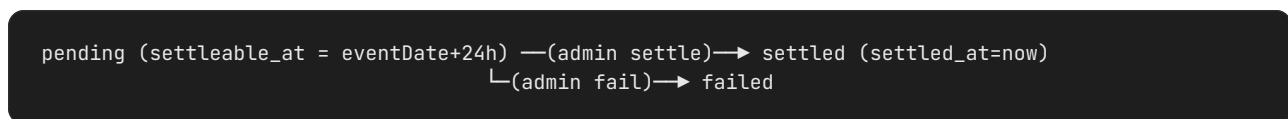
20.2 QUOTATION STATES



20.3 PAYMENT STATUS



20.4 PAYOUT STATUS



Next: `04-NFR-Non-Functional-Requirements.md` for measurable quality attributes, and `05-Use-Cases-and-User-Stories.md` for scenario-level acceptance.